

SIA Personal Financial Assistant

Module-by-module production execution plan

Decision in one line

Build SIA as a grounded financial tool agent: the model plans bounded read-only queries, BALENISA computes every financial fact, and SIA answers only from validated facts. Do not use RAG for live totals, budgets, forecasts, or transaction analytics.

Document	Value
Status	Proposed - implement only after each module gate is approved
Repository	D:\Projects\Personal\Expense\expense-manager
Verified against	Current backend SIA routing, query, provider, controller, session, idempotency, grounding, and frontend submission paths
Primary outcome	Natural-language answers across the user's financial data without sacrificing user isolation, correctness, or backward compatibility
Change policy	Minimal surface; no unrelated refactor; stable /sia/ask contract

1. Verified starting point

The plan below follows the repository's actual execution path, not a hypothetical redesign.

Stage	Current behavior	Keep / change
Entry	SiaPanel -> useSiaConversation -> POST /sia/ask	Keep the UI and HTTP contract
Boundary	Authentication, readiness, rate limiting, request validation	Keep unchanged
Reliability	Session ownership, request reservation, replay, routing checkpoint, answer-ready recovery	Keep unchanged; extend checkpoint compatibility
Fast path	Regex classifier maps known questions to report-backed explanation intents	Keep; correct category lookup misclassification
Fallback	LLM router emits QueryPlan v1 as prompt-only JSON	Replace output transport with provider-enforced structured output
Execution	financialQueryService performs aggregate-only, user-scoped reads	Keep boundary; add only bounded capabilities needed by plans
Composition	semanticPipeline executes plan.metrics[0] only	Change to execute every validated query
Answer	Direct lookups are deterministic; prose answers require cited FactSets	Keep and generalize
Client errors	Different semantic failures collapse into the same 422 message	Add stable error codes without breaking existing fields

Confirmed defect behind the screenshot

A natural category-total question can be claimed by CURRENT_SPENDING_SUMMARY, whose context contains only the overall total. If it reaches semantic routing instead, malformed or fenced provider JSON fails strict parsing and is reported as an unsupported question. Both paths can make a valid data question fail or answer the wrong scope.

2. Target architecture

One public request path, two safe planning paths, one authoritative data boundary.

Step	Module	Responsibility
1	Question boundary	Authenticate, validate length/IDs, reserve idempotency key, resolve owned session
2A	Deterministic fast path	Recognize proven intents and direct exact patterns; never guess category totals as overall totals
2B	Semantic planner	Provider returns a schema-constrained QueryPlan v2; no financial data is sent during planning
3	Plan validator	Fail closed on unknown keys, metrics, operators, periods, filters, query counts, or unsafe combinations
4	Financial tools	Execute authenticated, user-scoped, aggregate-only queries; never expose Mongo identifiers or raw models
5	FactSet	Normalize results into bounded facts with source, period, unit, and stable fact IDs
6A	Deterministic renderer	Use for simple totals, counts, category lookups, breakdowns, and no-data responses
6B	Grounded synthesizer	Use once for multi-fact explanations; accept only structured answer + citedFactIds
7	Validation + persistence	Reject uncited money/percent claims, persist one completed turn, return stable contract

Why not RAG for phase 1?

RAG retrieves text chunks by similarity. It cannot be the source of truth for exact user totals, date filters, categories, budgets, or comparisons. Those values must come from database tools. RAG is valuable later for unstructured documents, product help, policy explanations, and user-uploaded statements - never as the calculator for live personal finance.

Architecture invariants

- The model never chooses a userId; authenticated server context supplies it.
- The model never emits Mongo queries, field paths, collection names, or executable code.
- All time periods are locally resolved in APP_TIME_ZONE with start-inclusive/end-exclusive bounds.
- All plans and outputs are bounded before database or provider execution.
- Simple numeric answers do not pay for a second LLM call.
- No model-written financial figure reaches the UI unless it exists in a cited FactSet.

3. Stepwise delivery map

Proceed in order. Each gate produces a deployable, testable checkpoint.

Phase	Modules	Outcome	Go / no-go gate
A - Correctness foundation	0-2	Regression corpus, classifier fix, QueryPlan v2 contract	All old tests pass; category queries route correctly; v1 checkpoints remain valid
B - Reliable planning	3-4	Provider-native structured outputs and typed router outcomes	OpenAI/Gemini/Groq adapter contract tests pass; malformed output cannot become 422 unsupported
C - Complete execution	5-6	Bounded multi-query execution and FactSet merge	Every planned query executes once; caps, dedupe, and user scoping hold
D - Grounded intelligence	7-8	Deterministic or cited answers with honest errors	No uncited monetary claim; provider failures return retryable 503
E - Product completion	9-10	Frontend messaging, security/performance tests, provider smoke tests	Exact screenshot query and approved natural-language matrix pass end to end
Later	11	Optional document RAG	Separate approval; never participates in numeric calculations

Recommended commit sequence

- Commit 1: test corpus + classifier correction
- Commit 2: QueryPlan v2 + v1 compatibility
- Commit 3: structured-output provider adapters + router
- Commit 4: financial tool extensions + multi-query executor
- Commit 5: grounded synthesis + response validation
- Commit 6: controller error taxonomy + frontend handling
- Commit 7: end-to-end, adversarial, performance, and provider smoke verification

4. Module plans - foundation and planning

MODULE 0 Baseline behavior and regression corpus

Area	Plan
Files	backend/tests/sia.intentClassifier.test.js; sia.semanticRouter.test.js; sia.semanticPipeline.test.js; sia.ask.test.js; frontend SIA conversation tests
Objective	Freeze valid behavior before production edits.
Implementation	Add table-driven questions covering exact screenshot wording, tense/grammar variants, category totals, overall totals, income, budget, comparisons, follow-ups, unsupported requests, and provider failures. Assert route, plan, query calls, status, error code, and saved turn.
Depends on	None.
Acceptance gate	The new failing cases reproduce the current defect while the established test suite stays green.
Rollback	Test-only commit can be reverted without runtime effect.

MODULE 1 Deterministic intent boundary

Area	Plan
Files	backend/sia/intentClassifier.js; backend/tests/sia.intentClassifier.test.js
Objective	Prevent direct category lookup language from being swallowed by the overall spending summary intent.
Implementation	Tighten CURRENT_SPENDING_SUMMARY to require overall/total semantics without a named category. Preserve all existing explanation intents and their ordering. Let unambiguous category lookup questions reach semantic planning unless an exact deterministic category route is intentionally added.
Depends on	Module 0 corpus.
Acceptance gate	Overall totals still classify; category totals do not classify as overall; all prior intent tests pass.
Rollback	Revert the isolated classifier rule; semantic routing remains available.

MODULE 2 QueryPlan v2 contract and compatibility

Area	Plan
Files	backend/sia/queryPlan.js; backend/tests/sia.queryPlan.test.js; backend/tests/sia.idempotencyService.checkpoint.test.js; backend/tests/sia.sessionService.planSummary.test.js
Objective	Represent multi-part financial questions without trusting model-authored database logic.
Implementation	Introduce version 2 with queries[1..5]. Each query contains one metric, operation, period, optional comparisonPeriod/categoryFilter/grouping, and responseMode. Retain a validator/normalizer for version 1 checkpoints and convert v1 to the internal v2 execution form. Reject unknown keys, duplicate queries, unsupported combinations, ranges over 366 days, and unsafe category filters.
Depends on	Module 1; existing periodResolver and allowlists.
Acceptance gate	v1 and v2 fixtures normalize deterministically; invalid plans fail closed; persisted v1 retries do not call the router again.
Rollback	Keep v1 as the active emitted version while leaving the new validator inert.

5. Module plans - provider and router reliability

MODULE 3

Provider structured-output adapters

Area	Plan
Files	backend/sia/llmService.js; backend/tests/sia.llmService.openai.test.js; sia.llmService.gemini.test.js; sia.llmService.groq.test.js; sia.llmService.test.js
Objective	Make routing and synthesis machine-readable by contract, not by prompt hope.
Implementation	Add an optional structured schema/tool mode to askLlm while preserving the current text mode and return shape for report-backed intents. Map the same local JSON schema to each provider's supported structured-output/tool mechanism. Normalize the tool/structured payload locally. Classify timeout, HTTP, network, malformed structured output, and empty output with existing safe provider errors.
Depends on	Module 2 schema.
Acceptance gate	Provider request-body snapshots and extraction tests pass for all three adapters; text callers remain byte-for-byte compatible at the public service boundary.
Rollback	Disable structured mode and retain existing text adapters; no controller contract changes required.

MODULE 4

Semantic router with typed outcomes

Area	Plan
Files	backend/sia/semanticRouter.js; backend/tests/sia.semanticRouter.test.js
Objective	Separate true unsupported questions from routing infrastructure failures.
Implementation	Request QueryPlan v2 through llmService structured mode. Keep the static capability catalog, bounded prior-plan summary, and calendar guidance. Return one of: planned, clarification, unsupported, provider_failed, malformed_output. Validate locally even after provider schema enforcement. Never send financial values to the router.
Depends on	Modules 2-3.
Acceptance gate	A valid plan survives; unsupported is deliberate; provider timeout/malformed output is not mislabeled as unsupported; one router call maximum.
Rollback	Router can continue emitting v1 and use the existing pipeline while structured mode is feature-gated internally.

Provider rule

Structured output improves transport reliability; it does not grant trust. Every provider result still passes the local QueryPlan validator before any financial tool runs.

6. Module plans - financial tools and orchestration

MODULE 5 Financial query capability layer

Area	Plan
Files	backend/sia/financialQueryService.js; backend/tests/sia.financialQueryService.test.js
Objective	Supply the minimum authoritative facts needed for broad personal-finance questions.
Implementation	Preserve current totals/counts/category/budget/net-cash-flow methods. Add only approved bounded aggregates such as grouped monthly series and metric comparison. If item-level questions are approved, return at most a small safe projection (date, category, amount, user-visible name) and answer deterministically; never return IDs, descriptions, ML fields, or an unbounded list. Keep literal category matching and 366-day history cap.
Depends on	Module 2 metrics and periods; existing Mongo indexes.
Acceptance gate	Every query includes authenticated userId and bounded date range; cross-user fixtures return no data; caps and sort order are deterministic.
Rollback	New methods are additive and can be removed independently.

MODULE 6 Multi-query semantic pipeline

Area	Plan
Files	backend/sia/semanticPipeline.js; backend/tests/sia.semanticPipeline.test.js; backend/tests/sia.factSet.test.js
Objective	Execute every validated part of a question exactly once and produce one bounded fact set.
Implementation	Replace the plan.metrics[0] assumption with normalized query iteration. Dedupe equivalent queries. Resolve periods locally. Execute independent reads with a small concurrency cap; preserve deterministic ordering. Merge facts under per-query labels and enforce total fact/category limits. Route existing report-backed explanation intents through their unchanged handler. Use no answer call for direct lookups.
Depends on	Modules 2, 4, 5.
Acceptance gate	Two-metric questions execute both metrics; duplicates execute once; partial no-data is represented honestly; limits prevent unbounded work; existing single-metric answers remain unchanged.
Rollback	Normalize v2 back to a one-query path behind an internal flag during rollout.

7. Module plans - grounding, API behavior, and UI

MODULE 7 Answer composition and validation

Area	Plan
Files	backend/sia/semanticPipeline.js; backend/sia/responseValidator.js; backend/tests/sia.responseValidator.citedAnswer.test.js; backend/tests/sia.semanticPipeline.test.js
Objective	Answer useful multi-fact questions without model-authored calculations or invented values.
Implementation	Render simple totals, counts, breakdowns, and comparisons in backend code. For genuine explanation/synthesis, call the LLM once with the bounded FactSet and require structured {answer,citedFactIds}. Validate citation existence, every currency/percentage claim, temporal framing, comparison/forecast permissions, identifier leakage, unsafe advice, fraud claims, and empty output. Prefer server-calculated derived values over asking the model to calculate.
Depends on	Module 6 FactSet.
Acceptance gate	Injected amounts, uncited facts, raw IDs, unsupported certainty, and advice are rejected; approved answers cite every claimed numeric fact.
Rollback	Fall back to deterministic summaries for affected plans.

MODULE 8 Controller outcomes and persistence

Area	Plan
Files	backend/Controllers/SiaControllers/ask.js; backend/tests/sia.ask.*.test.js; backend/tests/sia.idempotencyService.checkpoint.test.js
Objective	Return honest, retryable errors and persist exactly one consistent conversation turn.
Implementation	Preserve response fields and status behavior where valid. Add stable error code SIA_UNSUPPORTED_QUESTION for deliberate out-of-scope 422 responses. Treat router/provider/structured-output failures as generic retryable 503. Keep clarification/no-data as 200. Save normalized plan checkpoints before execution; replay completed responses; never append failed turns; retain non-disclosing session ownership behavior.
Depends on	Modules 4, 6, 7; existing idempotency/session services.
Acceptance gate	422 occurs only for deliberate unsupported/prohibited questions; transient failures are 503; concurrent same-key requests produce one provider workflow and one saved turn.
Rollback	New error code is additive; existing message and status fields remain usable by old clients.

MODULE 9 Frontend conversation behavior

Area	Plan
Files	frontend/src/components/sia/useSiaConversation.js; frontend/src/components/sia/SiaPanel.js; focused SIA tests
Objective	Show the correct recovery action without redesigning the panel.
Implementation	Read the optional server error code. Show Retry for 503/network failures, a concise unsupported message for SIA_UNSUPPORTED_QUESTION, clarification options for 200 clarification, and existing no-data responses unchanged. Keep launcher, sessions, history, voice, and styling untouched.
Depends on	Module 8 additive error code.
Acceptance gate	A provider outage never says the question is unsupported; an unsupported question does not encourage useless retry; old responses still render.
Rollback	Old message-based handling remains a safe fallback when code is absent.

8. Module plans - production hardening and optional RAG

MODULE 10

Security, performance, and release verification

Area	Plan
Files	Existing SIA adversarial tests plus focused router/pipeline/controller/frontend tests; no production refactor
Objective	Prove the new flexibility does not weaken privacy, integrity, cost, or latency controls.
Implementation	Add prompt-injection, cross-user, oversized plan, unsafe category, excessive date range, duplicate query, provider malformed-output, concurrent idempotency, and stale-checkpoint tests. Measure router-only, deterministic, and synthesis paths. Confirm query counts and indexes with representative data. Run one configured-provider smoke test in a non-production account without logging prompts or credentials.
Depends on	Modules 0-9.
Acceptance gate	All focused and regression suites pass; no secret/ID leakage; bounded query/provider calls; p95 targets agreed before release; rollback exercised.
Rollback	Roll back commits in reverse order; database migration is not required.

MODULE 11

Optional document RAG - separate future work

Area	Plan
Files	New modules only after separate design approval; no phase-1 file changes
Objective	Answer questions about unstructured knowledge while keeping live finance authoritative.
Implementation	Use RAG only for BALENISA help, financial-literacy content, policies, or user-uploaded statements/documents. Enforce per-user namespaces, document lifecycle deletion, chunk provenance, retrieval limits, and citation display. If a question mixes documents with live data, retrieve text and execute financial tools separately, then synthesize from both labeled sources. Never derive totals from vector retrieval.
Depends on	Production stability of modules 0-10; privacy/retention decision.
Acceptance gate	Document answers cite source chunks; deleting a document removes all embeddings; numeric live-data tests remain tool-backed.
Rollback	Feature remains absent; SIA's financial assistant capability is complete without it.

9. QueryPlan v2 - internal contract

Recommended normalized shape

```
{
  "version": 2,
  "outcome": "supported",
  "queries": [
    { "metric": "CATEGORY_TOTAL", "operation": "LOOKUP",
      "period": { "type": "CURRENT_MONTH" },
      "categoryFilter": "Food", "grouping": "NONE" }
  ],
  "responseMode": "DETERMINISTIC"
}
```

Rule	Limit / behavior	Reason
Plan version	Accept v1 and v2; emit v2 after rollout	Backward-compatible idempotency checkpoints and saved plan summaries
Queries	1 to 5	Bound cost and latency
One metric per query	Required	Eliminates metrics[0] ambiguity
Period	Existing enum; max 366 days	Reuse periodResolver and history cap
Category filter	Max 60 chars; validated literal	Prevents operators, paths, regex abuse, and ReDoS
Comparison	Explicit second period	No inferred hidden comparison
Response mode	DETERMINISTIC or PROSE	Controls answer-call cost and grounding behavior
Unknown keys	Reject	Fail closed against provider drift
Equivalent queries	Canonicalize and dedupe	Avoid duplicate database work

Example: answer the screenshot question

Input	Plan	Execution	Answer path
Can you my current month total spending on food?	CATEGORY_TOTAL / CURRENT_MONTH / Food	getCategoryTotal(authenticated userId, resolved period, Food)	Deterministic: You spent [computed amount] on Food this month.

10. Failure contract and user experience

Condition	HTTP	Stable code / body	UI action	Persist turn?
Valid answer	200	Existing answer contract	Render answer	Yes
Clarification needed	200	clarification prompt + options	Render choices	No answered turn until resolved
No matching data	200	Existing honest no-data answer	Render no-data	Complete request; preserve existing policy
Deliberately unsupported	422	SIA_UNSUPPORTED_QUESTION + existing message	Explain scope; no Retry emphasis	No
Prohibited request	422	SIA_UNSUPPORTED_QUESTION	Explain scope	No
Router timeout/network	503	Existing generic unavailable body	Retry	No
Malformed structured output	503	Existing generic unavailable body	Retry	No
Answer grounding rejected	503	Existing generic unavailable body	Retry	No
Idempotency conflict/in progress	409	Existing contract	Existing handling	No duplicate
Foreign/missing session	404	Existing non-disclosing message	Existing handling	No
Rate limited	429	Existing limiter contract	Wait and retry	No

Compatibility rule

Do not remove or rename current response fields. Error code is additive. Existing frontend and older clients can continue using status + message while the new UI uses code for precise recovery behavior.

11. Verification matrix

Layer	Required checks	Release blocker
Classifier	Current/previous period, overall vs category, grammar variants, existing eight intents	Any category query classified as overall summary
Plan schema	v1 compatibility, v2 valid/invalid, caps, unknown keys, periods, filters, duplicates	Any unvalidated key reaches execution
Provider adapters	Request schema and extraction for OpenAI/Gemini/Groq; timeout/network/malformed output	Provider prose accepted as a plan
Financial tools	User scoping, exact date bounds, zero vs no-data, case-insensitive exact category, 366-day cap	Cross-user result or unbounded query
Pipeline	All queries execute, dedupe, bounded concurrency, partial no-data, no unnecessary answer call	Only first metric executed or duplicate provider call
Grounding	Citations, monetary/percentage claims, identifiers, advice, fraud/certainty/temporal guards	Uncited financial value reaches response
Controller	422 vs 503 taxonomy, idempotency replay/resume/concurrency, session ownership	Failed answer persisted or repeated provider charge
Frontend	Answer, no-data, clarification, unsupported, retryable failure, legacy response	Provider outage shown as unsupported
End to end	Screenshot wording plus at least 30 approved natural-language questions	Wrong period/category/metric answer
Operational	Safe logs, no credentials/prompts/IDs, latency and provider-call counters	Sensitive value in log or unbounded cost

Core end-to-end question set

- How much did I spend on Food this month?
- What was my total spending last month?
- Compare my income and expenses this month.
- Which category was highest in the previous quarter? (clarify or unsupported if quarter is not approved)
- How much budget is left for August 2026?
- Why is my forecast high?
- What about last month? (follow-up using bounded prior plan summary)
- Delete my Food expense. (unsupported because SIA is read-only)

12. Exact change surface

Production files that may change

File	Required reason
backend/sia/intentClassifier.js	Correct overall-vs-category classification boundary
backend/sia/queryPlan.js	Versioned multi-query contract and v1 normalization
backend/sia/llmService.js	Optional provider-native structured output/tool mode
backend/sia/semanticRouter.js	Typed router outcomes and QueryPlan v2 request
backend/sia/financialQueryService.js	Only approved missing bounded aggregate tools
backend/sia/semanticPipeline.js	Multi-query execution, dedupe, deterministic/prose routing
backend/sia/responseValidator.js	Multi-FactSet citation and claim validation
backend/Controllers/SiaControllers/ask.js	Error taxonomy and normalized checkpoint handling
frontend/src/components/sia/useSiaConversation.js	Optional error-code aware behavior
frontend/src/components/sia/SiaPanel.js	Only if a dedicated unsupported/retry state requires rendering

Files and domains that must remain unchanged

- Expense, income, budget, report, forecast, anomaly, and insights CRUD/business logic
- Mongo schemas and existing stored financial data
- Authentication, rate limiter, readiness, voice recording/transcription, launcher, history, and CSS
- Session ownership and idempotency algorithms except additive plan-version compatibility
- Public SIA routes and the existing /sia/ask request fields
- Existing report-backed contextBuilder pipeline for health, anomaly, forecast, and risk explanations

Scope checkpoint

If implementation requires a file outside this list, stop and re-audit before editing. A new requirement must justify the expanded surface; convenience refactoring is not sufficient.

13. Operational controls and trade-offs

Dimension	Decision	Trade-off
Correctness	Server computes facts and derived values	More backend capability code; far lower hallucination risk
Latency	One router call; zero answer calls for simple lookups; one for synthesis	Complex questions cost two provider calls
Cost	Cap plans at 5 queries; dedupe; bounded facts/history	Some broad questions require clarification
Privacy	Router sees question + capability catalog only; answer model sees bounded facts	Provider still sees the user's question unless a local model is selected
Availability	Provider failures are retryable 503; deterministic fast path still uses current provider for existing explanation intents	Free-form routing requires provider availability
Compatibility	Accept QueryPlan v1 checkpoints; additive response codes	Temporary dual-version code until old checkpoints age out
Maintainability	Reuse periodResolver, financialQueryService, FactSet, validators	Avoids a generic agent framework but keeps explicit capability work
RAG	Defer to a separate document-knowledge phase	No document Q&A; in phase 1; live financial Q&A; is stronger and safer

Monitoring after rollout

- Count outcomes: deterministic answer, synthesized answer, clarification, no-data, unsupported, provider failure, grounding rejection.
- Measure router latency, answer latency, database query count, and provider calls per request without logging question text or facts.
- Alert on spikes in malformed structured output, grounding rejection, 503 rate, idempotency conflicts, and p95 latency.
- Review anonymized intent/metric counters to decide which capability to add next; do not retain raw personal questions solely for analytics.

14. Definition of done and next action

SIA is production-ready for this scope only when all statements are true:

- 01 The exact reported Food/current-month question returns the category total for the authenticated user.
- 02 Natural phrasing does not depend on regex-only intent coverage.
- 03 Every executed query came from a locally validated, bounded plan.
- 04 Every financial read is authenticated, user-scoped, and date-bounded.
- 05 All plan queries execute; the pipeline never silently uses only the first metric.
- 06 Simple numeric answers are deterministic; synthesized answers cite every numeric fact.
- 07 Provider/structured-output failures are distinguishable from deliberate unsupported questions.
- 08 Idempotent retries do not duplicate provider calls, sessions, or messages.
- 09 Existing SIA explanation intents, sessions, voice, history, auth, and financial modules pass regression tests.
- 10 No database migration, public route change, or unrelated refactor is included.

Recommended next step

Approve Phase A only: Module 0 regression corpus, Module 1 classifier boundary, and Module 2 QueryPlan v2 design. Implement and review that checkpoint before changing provider adapters or database execution.

Reference basis

Repository modules verified for this plan: `intentClassifier.js`, `queryPlan.js`, `semanticRouter.js`, `semanticPipeline.js`, `financialQueryService.js`, `llmService.js`, `responseValidator.js`, `Controllers/SiaControllers/ask.js`, `useSiaConversation.js`, `SiaPanel.js`, and focused SIA tests. External capability references: OpenAI Function Calling and Structured Outputs; Google Gemini Tools; Groq Local Tool Calling and Structured Outputs.

Official references: developers.openai.com/api/docs/guides/function-calling | developers.openai.com/api/docs/guides/structured-outputs | ai.google.dev/gemini-api/docs/tools | console.groq.com/docs/tool-use/local-tool-calling | console.groq.com/docs/structured-outputs